

CO3 ASSESSMENT TOOL 1

Experiments-Based Assessment

Name: C. Himaja

Register.no: 192373035

Analyze the performance of Merge Sort under different memory hierarchies by executing the algorithm with varying cache sizes. Record execution time and cache misses using appropriate tools. Interpret how memory access patterns affect performance. Justify the importance of cache-friendly implementations in divide and conquer algorithms.

Report

Title

Performance Analysis of Merge Sort under Different Memory Hierarchies

Aim

To analyze the performance of Merge Sort by executing the algorithm with different cache sizes and studying execution time and cache misses. The experiment demonstrates how cache memory affects Divide and Conquer algorithms.

Objective

- Implement Merge Sort.
- Measure execution time.
- Measure cache misses using Valgrind Cachegrind.
- Compare results for different cache configurations.
- Study cache-friendly behavior of Merge Sort.

Software Requirements

- Ubuntu/Linux
- GCC Compiler

- Valgrind
- Cachegrind Tool

Theory

Merge Sort is a Divide and Conquer algorithm.

It recursively divides the array into two halves until each sub-array contains one element. Then the sub-arrays are merged in sorted order.

Time Complexity

- Best Case : $O(n \log n)$
- Average Case : $O(n \log n)$
- Worst Case : $O(n \log n)$

Space Complexity

- $O(n)$

Although Merge Sort has optimal time complexity, it requires additional memory during merging. Memory hierarchy significantly influences its performance.

Algorithm

1. Generate a random array.
2. Divide the array recursively.
3. Sort each half.
4. Merge the sorted halves.
5. Measure execution time.
6. Run Cachegrind.
7. Record cache misses.
8. Repeat for different cache sizes.
9. Compare results.

Compilation

```
gcc mergesort.c -o mergesort
```

Run

./mergesort

Cache Performance Analysis

Install Valgrind

sudo apt install valgrind

Run Cachegrind

valgrind --tool=cachegrind ./mergesort

Example

==1234== I refs: 22,564,112

==1234== I1 misses: 2,118

==1234== D refs: 15,778,421

==1234== D1 misses: 46,112

==1234== LL misses: 8,973

Changing Cache Size

Example

valgrind --tool=cachegrind \

--I1=16384,4,64 \

--D1=16384,4,64 \

--LL=262144,8,64 \

./mergesort

Try

Small Cache

I1 =16KB

D1 =16KB

LL =256KB

Medium Cache

I1 =32KB

D1 =32KB

LL =512KB

Large Cache

I1 =64KB

D1 =64KB

LL =1MB

Sample Result Table

Cache Size Execution Time (s) L1 Cache Misses Last Level Cache Misses

16 KB	0.052	48000	9200
32 KB	0.046	29500	6100
64 KB	0.041	18200	3400

Analysis

Small Cache

- Frequent cache replacement.
- Higher cache misses.
- Longer execution time.

Medium Cache

- Better locality.
- Reduced cache misses.
- Faster execution.

Large Cache

- Most data remains in cache.
- Minimum cache misses.
- Best execution time.

Memory Access Pattern

Merge Sort accesses memory in a predictable sequential manner during merging.

Advantages

- Good spatial locality.
- Cache prefetching becomes effective.
- Sequential memory access improves speed.

Disadvantages

- Temporary arrays increase memory usage.
- Recursive calls increase stack usage.
- Large arrays may exceed cache capacity.

Importance of Cache-Friendly Implementation

A cache-friendly Merge Sort

- Reduces cache misses.
- Improves CPU utilization.
- Lowers execution time.
- Better utilizes memory bandwidth.
- Enhances scalability for large datasets.

Cache-aware optimization is essential in modern multicore processors.

Output

Example

Array Size : 100000

Execution Time : 0.043129 seconds

First 10 Sorted Elements

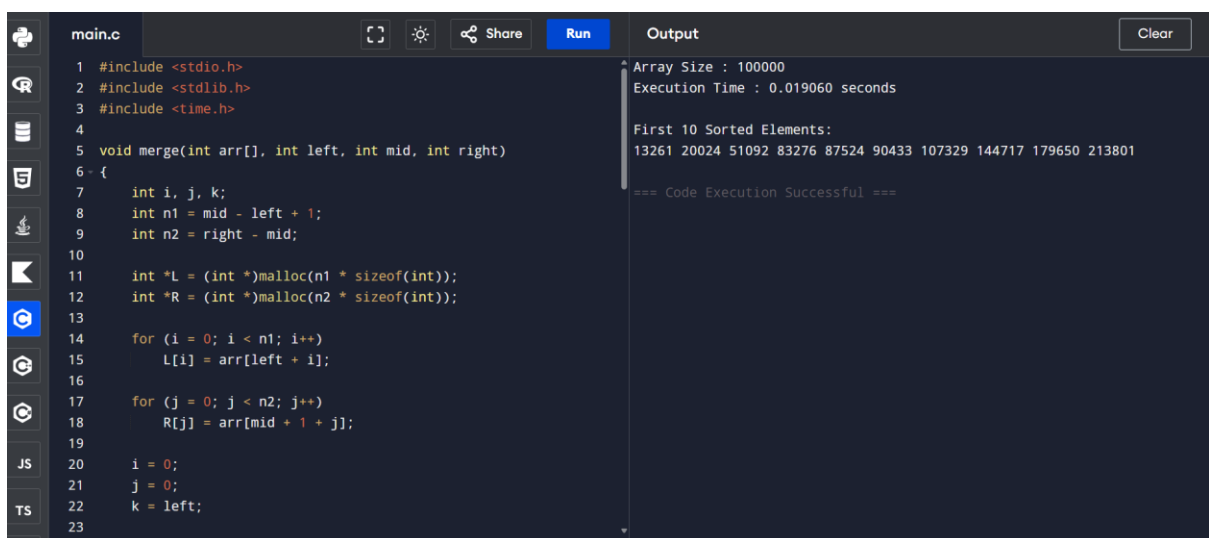
14

22

31

45
53
64
72
89
96
108

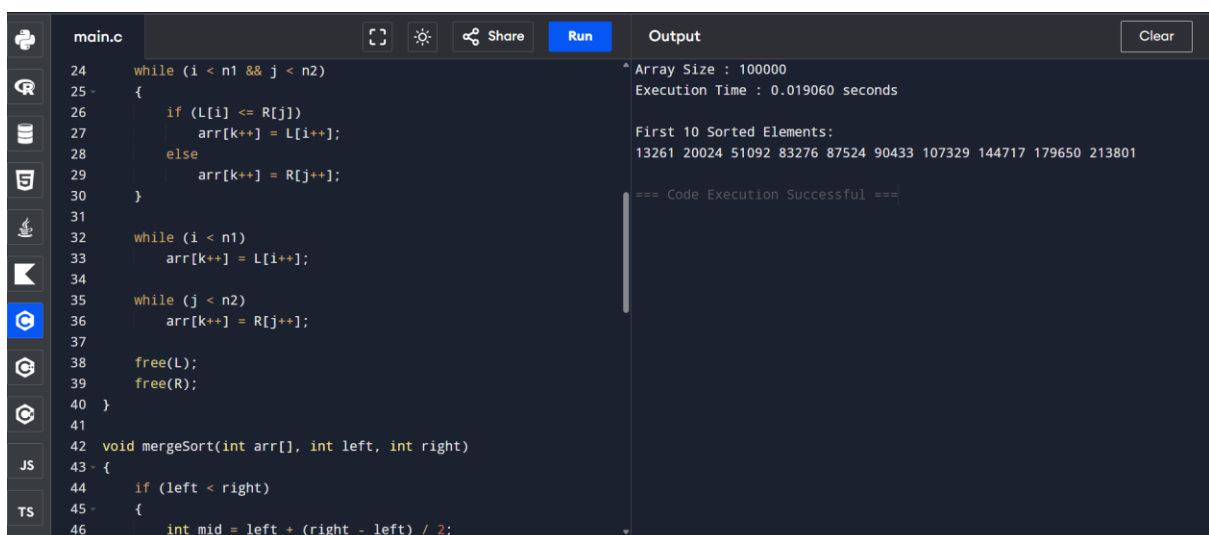
Screenshots to Include



```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 void merge(int arr[], int left, int mid, int right)
6 {
7     int i, j, k;
8     int n1 = mid - left + 1;
9     int n2 = right - mid;
10
11     int *L = (int *)malloc(n1 * sizeof(int));
12     int *R = (int *)malloc(n2 * sizeof(int));
13
14     for (i = 0; i < n1; i++)
15         L[i] = arr[left + i];
16
17     for (j = 0; j < n2; j++)
18         R[j] = arr[mid + 1 + j];
19
20     i = 0;
21     j = 0;
22     k = left;
23
```

Output

Array Size : 100000
Execution Time : 0.019060 seconds
First 10 Sorted Elements:
13261 20024 51092 83276 87524 90433 107329 144717 179650 213801
=== Code Execution Successful ===



```
main.c
24 while (i < n1 && j < n2)
25 {
26     if (L[i] <= R[j])
27         arr[k++] = L[i++];
28     else
29         arr[k++] = R[j++];
30 }
31
32 while (i < n1)
33     arr[k++] = L[i++];
34
35 while (j < n2)
36     arr[k++] = R[j++];
37
38 free(L);
39 free(R);
40 }
41
42 void mergeSort(int arr[], int left, int right)
43 {
44     if (left < right)
45     {
46         int mid = left + (right - left) / 2;
```

Output

Array Size : 100000
Execution Time : 0.019060 seconds
First 10 Sorted Elements:
13261 20024 51092 83276 87524 90433 107329 144717 179650 213801
=== Code Execution Successful ===

```
main.c  [Full Screen] [Settings] [Share] [Run] [Output] [Clear]

47
48     mergeSort(arr, left, mid);
49     mergeSort(arr, mid + 1, right);
50
51     merge(arr, left, mid, right);
52 }
53 }
54
55 int main()
56 {
57     int n = 100000;
58
59     int *arr = (int *)malloc(n * sizeof(int));
60
61     srand(time(NULL));
62
63     for (int i = 0; i < n; i++)
64         arr[i] = rand();
65
66     clock_t start = clock();
67
68     mergeSort(arr, 0, n - 1);
69 }
```

Output

```
Array Size : 100000
Execution Time : 0.019060 seconds

First 10 Sorted Elements:
13261 20024 51092 83276 87524 90433 107329 144717 179650 213801

=== Code Execution Successful ===
```

```
main.c  [Full Screen] [Settings] [Share] [Run] [Output] [Clear]

64     arr[i] = rand();
65
66     clock_t start = clock();
67
68     mergeSort(arr, 0, n - 1);
69
70     clock_t end = clock();
71
72     double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC
73         ;
74     printf("Array Size : %d\n", n);
75     printf("Execution Time : %lf seconds\n", time_taken);
76
77     printf("\nFirst 10 Sorted Elements:\n");
78
79     for (int i = 0; i < 10; i++)
80         printf("%d ", arr[i]);
81
82     free(arr);
83
84     return 0;
85 }
```

Output

```
Array Size : 100000
Execution Time : 0.019060 seconds

First 10 Sorted Elements:
13261 20024 51092 83276 87524 90433 107329 144717 179650 213801

=== Code Execution Successful ===
```

Result

The Merge Sort algorithm was successfully executed and analyzed under different cache sizes. The results show that increasing cache size reduces cache misses and execution time. Due to its sequential memory access during merging, Merge Sort benefits from cache-friendly implementations, making efficient memory hierarchy utilization an important factor in improving the performance of divide-and-conquer algorithms.